

EDA and Model Evaluation Metrics

KJR

CONTENT

1. Exploratory Data Analysis

2. Model Evaluation

- Regularization in Machine Learning
- **Confusion Matrix**
- Model Performance Measuring Metrics

❑ Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an important step in data analysis where we explore, summarize, and visualize data to understand its structure, detect patterns, identify anomalies, test assumptions, and check relationships between variables before applying any machine learning or statistical models.

Exploratory Data Analysis (EDA)



EDA

Explore and summarize data before modeling



Distribution

Use histograms and box plots



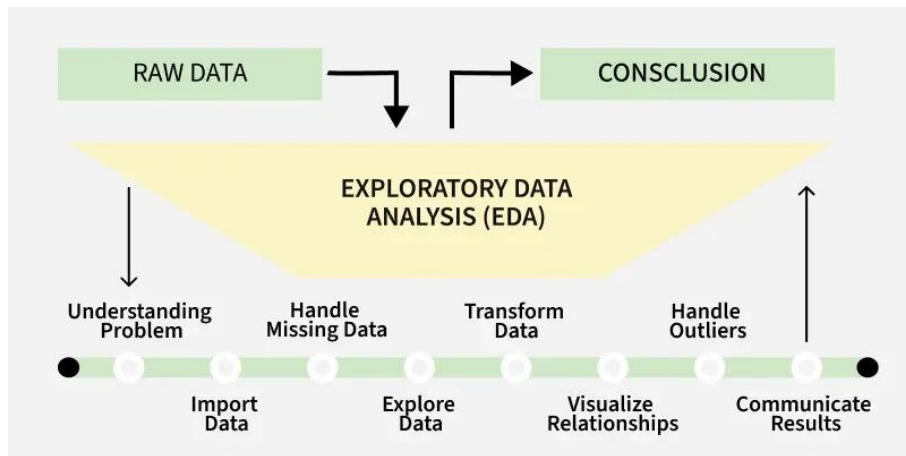
Relationships

Use scatter and pair plots



Spot Issues

Find outliers or missing data



Importance

- Provides a clear understanding of the dataset, including the number of features, data types and data distribution.
- Reveals patterns and relationships between different variables in the data.
- Identifies errors and outliers that may affect analysis.
- Highlights the most important features useful for building models.
- Supports selecting suitable modelling techniques for better results.

1. Univariate Analysis

Univariate analysis studies one variable at a time to understand its characteristics and distribution.

- **Histograms:** Show how data values are distributed.
- **Box plots:** Help detect outliers and show data spread.
- **Bar charts:** Used for categorical variables.

2. Bivariate Analysis

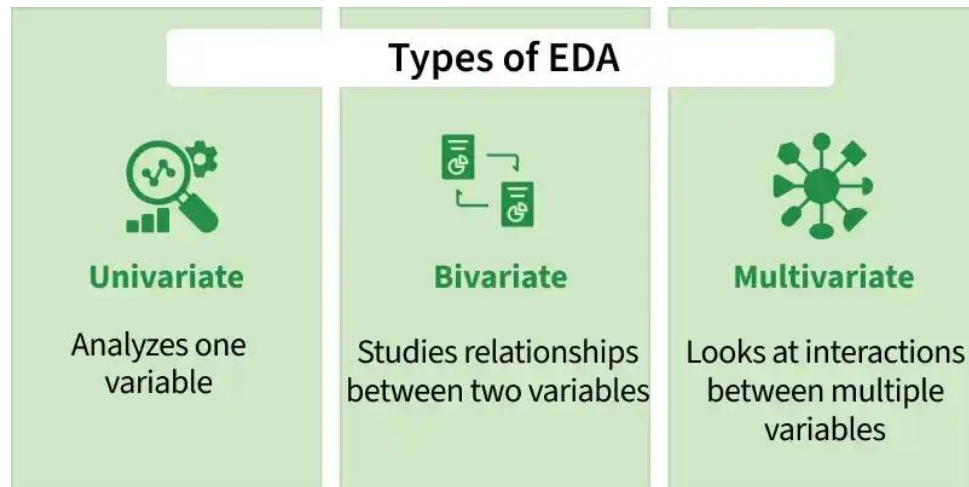
Bivariate analysis examines the relationship between two variables to understand how they interact or influence each other. Common techniques include:

- **Scatter plots:** Show the relationship between two numerical variables.
- **Correlation coefficient:** Measures the strength of the relationship between variables.
- **Cross-tabulation:** Displays the relationship between two categorical variables.
- **Line graphs:** Compare two variables over time to identify trends.
- **Covariance:** Shows how two variables change together.

3. Multivariate Analysis

Multivariate analysis studies three or more variables together to understand complex relationships within the dataset. Common techniques include:

- **Pair plots:** Show relationships between multiple variables at once.
- **Principal Component Analysis (PCA):** Reduces dimensionality while preserving important information.
- **Spatial analysis:** Analyzes geographical patterns using maps and location-based data.



❑ **Steps for Performing Exploratory Data Analysis**

01

**Understanding
the Problem
and the Data**

02

**Importing and
Inspecting the
Data**

03

**Handling
Missing Data**

04

**Exploring Data
Characteristics**

05

**Performing Data
Transformation**

06

**Visualizing
Relationship of
Data**

07

**Handling
Outliers**

08

**Communicate
Findings and
Insights**

EDA - Exploratory Data Analysis in Python

EDA is an essential step in data analysis that focuses on understanding patterns, relationships and distributions within a dataset using statistical methods and visualizations. Python libraries such as pandas, NumPy, plotly, matplotlib and seaborn make this process efficient and insightful. Some common EDA techniques are:

- Data Inspection:** Check the size of the dataset, how it is organized, the types of data it contains and basic summary values.
- Handling Missing and Duplicate Data:** Find and fix empty values or repeated rows to keep the data clean.
- Univariate Analysis:** Study one variable at a time to understand its distribution, trend and outliers.
- Bivariate Analysis:** Compare two variables to see how they are related.
- Multivariate Analysis:** Analyze three or more variables together to understand deeper relationships.

Key Steps for Exploratory Data Analysis (EDA)

Step 1: Importing Required Libraries

We need to install [Pandas](#), [NumPy](#), [Matplotlib](#) and [Seaborn](#) libraries in python to proceed further.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings as wr
wr.filterwarnings('ignore')
```

Step 2: Reading Dataset

Let's read the dataset using pandas.

Download the dataset from this [link](#)

```
df = pd.read_csv("/content/WineQT.csv")
print(df.head())
```

Output:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
0	7.4	0.70	0.00	1.9	0.076	
1	7.8	0.88	0.00	2.6	0.098	
2	7.8	0.76	0.04	2.3	0.092	
3	11.2	0.28	0.56	1.9	0.075	
4	7.4	0.70	0.00	1.9	0.076	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
0	11.0	34.0	0.9978	3.51	0.56	
1	25.0	67.0	0.9968	3.20	0.68	
2	15.0	54.0	0.9970	3.26	0.65	
3	17.0	60.0	0.9980	3.16	0.58	
4	11.0	34.0	0.9978	3.51	0.56	

	alcohol	quality	Id
0	9.4	5	0
1	9.8	5	1
2	9.8	5	2
3	9.8	6	3
4	9.4	5	4

Step 3: Analyzing the Data

1. df.shape(): This function is used to understand the number of rows (observations) and columns (features) in the dataset. This gives an overview of the dataset's size and structure.

df.shape

Output:

(1143, 13)

2. df.info(): This function helps us to understand the dataset by showing the number of records in each column, type of data, whether any values are missing and how much memory the dataset uses.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1143 entries, 0 to 1142
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	fixed acidity	1143 non-null	float64
1	volatile acidity	1143 non-null	float64
2	citric acid	1143 non-null	float64
3	residual sugar	1143 non-null	float64
4	chlorides	1143 non-null	float64
5	free sulfur dioxide	1143 non-null	float64
6	total sulfur dioxide	1143 non-null	float64
7	density	1143 non-null	float64
8	pH	1143 non-null	float64
9	sulphates	1143 non-null	float64
10	alcohol	1143 non-null	float64
11	quality	1143 non-null	int64
12	Id	1143 non-null	int64

```
dtypes: float64(11), int64(2)
```

```
memory usage: 116.2 KB
```

3. df.describe().T: This method gives a statistical summary of the DataFrame (Transpose) showing values like count, mean, standard deviation, minimum and quartiles for each numerical column. It helps in summarizing the central tendency and spread of the data.

df.describe().T

	count	mean	std	min	25%	50%	75%	max
fixed acidity	1143.0	8.311111	1.747595	4.60000	7.10000	7.90000	9.100000	15.90000
volatile acidity	1143.0	0.531339	0.179633	0.12000	0.39250	0.52000	0.640000	1.58000
citric acid	1143.0	0.268364	0.196686	0.00000	0.09000	0.25000	0.420000	1.00000
residual sugar	1143.0	2.532152	1.355917	0.90000	1.90000	2.20000	2.600000	15.50000
chlorides	1143.0	0.086933	0.047267	0.01200	0.07000	0.07900	0.090000	0.61100
free sulfur dioxide	1143.0	15.615486	10.250486	1.00000	7.00000	13.00000	21.000000	68.00000
total sulfur dioxide	1143.0	45.914698	32.782130	6.00000	21.00000	37.00000	61.000000	289.00000
density	1143.0	0.996730	0.001925	0.99007	0.99557	0.99668	0.997845	1.00369
pH	1143.0	3.311015	0.156664	2.74000	3.20500	3.31000	3.400000	4.01000
sulphates	1143.0	0.657708	0.170399	0.33000	0.55000	0.62000	0.730000	2.00000
alcohol	1143.0	10.442111	1.082196	8.40000	9.50000	10.20000	11.100000	14.90000
quality	1143.0	5.657043	0.805824	3.00000	5.00000	6.00000	6.000000	8.00000
Id	1143.0	804.969379	463.997116	0.00000	411.00000	794.00000	1209.500000	1597.00000

4. **df.columns.tolist():** This converts the column names of the DataFrame into a Python list making it easy to access and manipulate the column names.

```
df.columns.tolist()
```

Output:

```
['fixed acidity',  
'volatile acidity',  
'citric acid',  
'residual sugar',  
'chlorides',  
'free sulfur dioxide',  
'total sulfur dioxide',  
'density',  
'pH',  
'sulphates',  
'alcohol',  
'quality',  
'Id']
```

Step 4 : Checking Missing Values

df.isnull().sum(): This checks for missing values in each column and returns the total number of null values per column helping us to identify any gaps in our data.

```
df.isnull().sum()
```

Output:

	0
fixed acidity	0
volatile acidity	0
citric acid	0
residual sugar	0
chlorides	0
free sulfur dioxide	0
total sulfur dioxide	0
density	0
pH	0
sulphates	0
alcohol	0
quality	0
Id	0

dtype: int64

Step 5 : Checking for the duplicate values

df.duplicated().sum(): Returns the number of duplicate rows in the dataset.

```
df.duplicated().sum()
```

Output:

	0
fixed acidity	91
volatile acidity	135
citric acid	77
residual sugar	80
chlorides	131
free sulfur dioxide	53
total sulfur dioxide	138
density	388
pH	87
sulphates	89
alcohol	61
quality	6
Id	1143

dtype: int64

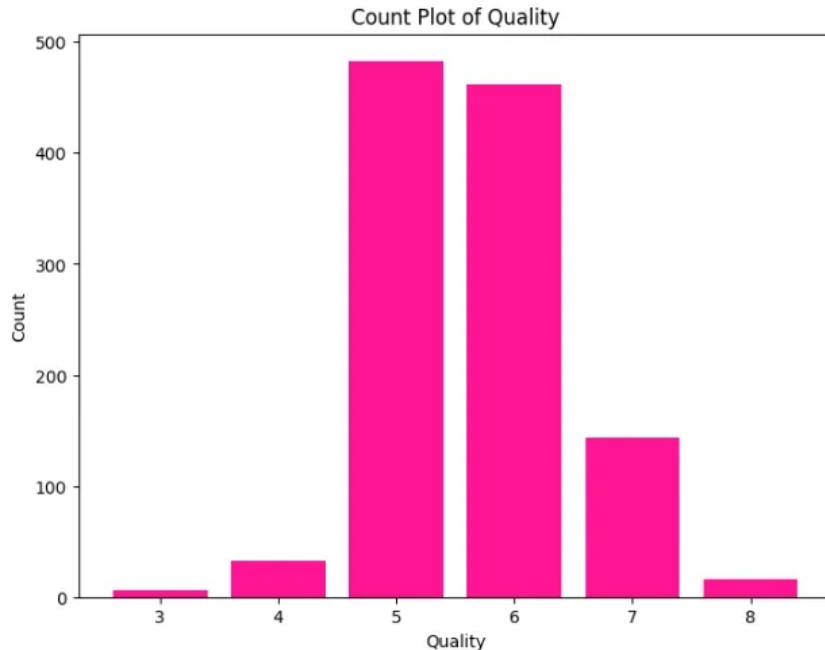
Step 6: Univariate Analysis

In [Univariate analysis](#) plotting the right charts can help us to better understand the data making the data visualization so important.

1. Bar Plot for evaluating the count of the wine with its quality rate.

```
quality_counts = df['quality'].value_counts()
```

```
plt.figure(figsize=(8, 6))
plt.bar(quality_counts.index, quality_counts, color='deeppink')
plt.title('Count Plot of Quality')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.show()
Output:
```



If you want to practice Bivariate and multivariate analysis

[Blog Link:](#)

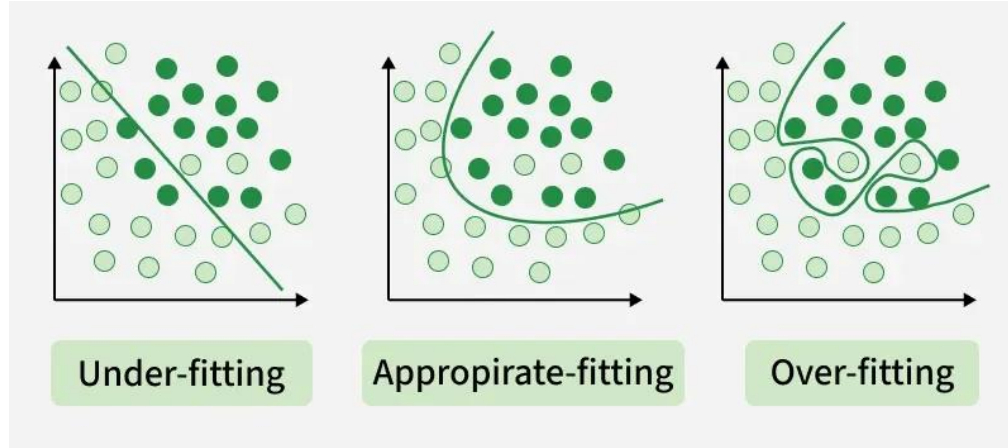
[EDA - Exploratory Data Analysis in Python – GeeksforGeeks](#)

Here, this count plot graph shows the count of the wine with its quality rate.

❑ Regularization in Machine Learning

Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data. By adding a penalty for complexity, regularization encourages simpler and more generalizable models.

- **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization:** Encourages simpler models that perform better on new, unseen data.



Benefits of Regularization

Now, let's see various benefits of regularization which are as follows:

- **Prevents Overfitting:** Regularization helps models focus on underlying patterns instead of memorizing noise in the training data.
- **Enhances Performance:** Prevents excessive weighting of outliers or irrelevant features helps in improving overall model accuracy.
- **Stabilizes Models:** Reduces sensitivity to minor data changes which ensures consistency across different data subsets.
- **Prevents Complexity:** Keeps model from becoming too complex which is important for limited or noisy data.
- **Handles Multicollinearity:** Reduces the magnitudes of correlated coefficients helps in improving model stability.
- **Promotes Consistency:** Ensures reliable performance across different datasets which reduces the risk of large performance shifts.

❏ Confusion Matrix in Machine Learning

Confusion matrix is a simple table used to measure how well a classification model is performing. It compares the predictions made by the model with the actual results and shows where the model was right or wrong. This helps you understand where the model is making mistakes so you can improve it. It breaks down the predictions into four categories:

- True Positive (TP)**: The model correctly predicted a positive outcome i.e the actual outcome was positive.
- True Negative (TN)**: The model correctly predicted a negative outcome i.e the actual outcome was negative.
- False Positive (FP)**: The model incorrectly predicted a positive outcome i.e the actual outcome was negative. It is also known as a Type I error.
- False Negative (FN)**: The model incorrectly predicted a negative outcome i.e the actual outcome was positive. It is also known as a Type II error.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Example 1: Agricultural Disease Detection

Imagine an AI model looking at images of crop leaves to detect a disease.

- Target (Positive):** Diseased Leaf

- Negative:** Healthy Leaf

Index	Actual	Predicted	Result
1	Diseased	Diseased	TP
2	Healthy	Healthy	TN
3	Diseased	Healthy	FN
4	Diseased	Diseased	TP
5	Healthy	Diseased	FP
6	Healthy	Healthy	TN
7	Diseased	Diseased	TP
8	Healthy	Healthy	TN
9	Diseased	Diseased	TP
10	Healthy	Healthy	TN

Summary Counts:

Actual Diseased Counts = 5

Actual Healthy Counts = 5

True Positive Counts = 4

False Positive Counts = 1

True Negative Counts = 4

False Negative Counts = 1

Confusion Matrix:

	Predicted: Diseased	Predicted: Healthy
Actual: Diseased	True Positive (TP = 4)	False Negative (FN = 1)
Actual: Healthy	False Positive (FP = 1)	True Negative (TN = 4)

Example 2: Confusion Matrix for Dog Image Recognition with Numbers

Index	1	2	3	4	5	6	7	8	9	10
Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

Confusion Matrix:

- Actual Dog Counts = 6
- Actual Not Dog Counts = 4
- True Positive Counts = 5
- False Positive Counts = 1
- True Negative Counts = 3
- False Negative Counts = 1

		Predicted	
		Dog	Not Dog
Actual	Dog	True Positive (TP =5)	False Negative (FN =1)
	Not Dog	False Positive (FP=1)	True Negative (TN=3)

Performance Metrics based on Confusion Matrix Data

Accuracy

Accuracy shows how many predictions the model got right out of all the predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Precision focus on the quality of the model's positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Recall measures how how good the model is at predicting positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score

F1-score combines precision and recall into a single metric to balance their trade-off.

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Specificity

It measures the ability of a model to correctly identify negative instances.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Type 1 error: It occurs when the model incorrectly predicts a positive instance but the actual instance is negative.

$$\text{Type 1 Error} = \frac{FP}{FP + TN}$$

Type 2 error: This occurs when the model fails to predict a positive instance even though it is actually positive.

$$\text{Type 2 Error} = \frac{FN}{TP + FN}$$

➤ **H.W. Find accuracy, precision, recall, f1-score, specificity, type 1 and 2 error for example 2.**

❑ Implementation of Confusion Matrix for Binary classification using Python

Step 1: Import the necessary libraries

```
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

Step 3: Compute the confusion matrix

•**confusion_matrix:** This function from sklearn.metrics computes the confusion matrix which is a table used to evaluate the performance of a classification algorithm. It compares actual and predicted to generate a matrix

```
import numpy as np from sklearn.metrics
import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

Step 2: Create the NumPy array for actual and predicted labels

- actual:** represents the true labels or the actual classification of the items. In this case it's a list of 10 items where each entry is either 'Dog' or 'Not Dog'.
- predicted:** represents the predicted labels or the classification made by the model.

```
actual = np.array(['Dog', 'Dog', 'Dog', 'Not Dog', 'Dog', 'Not Dog', 'Dog', 'Dog', 'Not Dog', 'Not Dog'])
predicted = np.array(['Dog', 'Not Dog', 'Dog', 'Not Dog', 'Dog', 'Dog', 'Dog', 'Dog', 'Not Dog', 'Not Dog'])
```

Step 3: Compute the confusion matrix

•**confusion_matrix:** This function from sklearn.metrics computes the confusion matrix which is a table used to evaluate the performance of a classification algorithm. It compares actual and predicted to generate a matrix

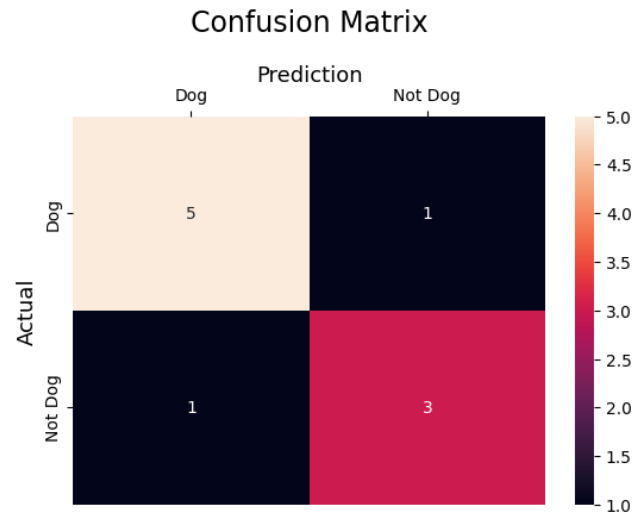
```
cm = confusion_matrix(actual, predicted)
```

Step 4: Plot the confusion matrix with the help of the seaborn heatmap

•**sns.heatmap**: This function from Seaborn is used to create a heatmap of the confusion matrix.

•**annot=True**: Display the numerical values in each cell of the heatmap.

```
sns.heatmap(cm,
            annot=True,
            fmt='g',
            xticklabels=['Dog', 'Not Dog'],
            yticklabels=['Dog', 'Not Dog'])
plt.ylabel('Actual', fontsize=13)
plt.title('Confusion Matrix', fontsize=17, pad=20)
plt.gca().xaxis.set_label_position('top')
plt.xlabel('Prediction', fontsize=13)
plt.gca().xaxis.tick_top()
plt.gca().figure.subplots_adjust(bottom=0.2)
plt.gca().figure.text(0.5, 0.05, 'Prediction', ha='center', fontsize=13)
plt.show()
```



Step 5: Classifications Report based on Confusion Metrics

```
print(classification_report(actual, predicted))
```

	precision	recall	f1-score	support
Dog	0.83	0.83	0.83	6
Not Dog	0.75	0.75	0.75	4
accuracy			0.80	10
macro avg	0.79	0.79	0.79	10
weighted avg	0.80	0.80	0.80	10

❑ Confusion Matrix For Multi-class Classification

In multi-class classification the confusion matrix is expanded to account for multiple classes.

- **Rows** represent the actual classes (ground truth).
- **Columns** represent the predicted classes.
- Each cell in the matrix shows how often a specific actual class was predicted as another class.

Confusion Matrix for Image Classification (Cat, Dog, Horse)

Actual/ Predicted	Predicted Cat	Predicted Dog	Predicted Horse
Actual Cat	Correct	Misclassified	Misclassified
Actual Dog	Misclassified	Correct	Misclassified
Actual Horse	Misclassified	Misclassified	Correct

Example -1: simple **multiclass classification confusion matrix example** for three classes: **Cat, Dog, and Horse**,

Index	1	2	3	4	5	6	7	8	9	10
Actual	Cat	Dog	Horse	Cat	Dog	Horse	Cat	Dog	Horse	Cat
Predicted	Cat	Dog	Dog	Horse	Dog	Horse	Cat	Cat	Horse	Cat
Result	Correct	Correct	Misclassified	Misclassified	Correct	Correct	Correct	Misclassified	Correct	Correct

Explanation of Misclassified Samples

- Index 3:** Actual = Horse, Predicted = Dog
- Index 4:** Actual = Cat, Predicted = Horse
- Index 8:** Actual = Dog, Predicted = Cat

Corresponding Confusion Matrix

Actual \ Predicted	Cat	Dog	Horse
Cat	3	0	1
Dog	1	2	0
Horse	0	1	2

➤ **H.W. Find accuracy, precision, recall, f1-score, specificity, type 1 and 2 error for cat, dog and horse class**

❖ Find accuracy, precision, recall, f1-score, specificity, type 1 and 2 error for cat class

Step 1: Find TP, TN, FP, FN for Cat

Treat:

- Cat = Positive class
- Dog + Horse = Negative class

TP (True Positive)

Actual Cat and Predicted Cat = 3

FN (False Negative)

Actual Cat but predicted something else = 1

FP (False Positive)

Predicted Cat but actually not Cat

Dog → Cat = 1

Horse → Cat = 0

So FP = 1

TN (True Negative)

Everything except Cat-related errors/correcs

Total = 10

$$TN = 10 - (TP + FP + FN)$$

$$TN = 10 - (3 + 1 + 1) = 5$$

Step 2: Accuracy

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ &= \frac{3 + 5}{10} \\ &= \frac{8}{10} = 0.8 \end{aligned}$$

Accuracy = **80%**

Step 3: Precision

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} \\ &= \frac{3}{3 + 1} \\ &= \frac{3}{4} = 0.75 \end{aligned}$$

Precision = **75%**

Step 4: Recall (Sensitivity)

$$\begin{aligned}\text{Recall} &= \frac{TP}{TP + FN} \\ &= \frac{3}{3 + 1} \\ &= 0.75\end{aligned}$$

Recall = **75%**

Step 5: F1-Score

$$\begin{aligned}F1 &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= \frac{2 \times 0.75 \times 0.75}{0.75 + 0.75} \\ &= 0.75\end{aligned}$$

F1-score = **75%**

Step 6: Specificity

$$\begin{aligned}\text{Specificity} &= \frac{TN}{TN + FP} \\ &= \frac{5}{5 + 1} \\ &= \frac{5}{6} = 0.8333\end{aligned}$$

Specificity ≈ **83.33%**

Step 7: Type I Error

Type I Error = False Positive Rate

$$\begin{aligned}\text{Type I Error} &= \frac{FP}{FP + TN} \\ &= \frac{1}{1 + 5} \\ &= \frac{1}{6} = 0.1667\end{aligned}$$

Type I Error ≈ **16.67%**

Step 8: Type II Error

Type II Error = False Negative Rate

$$\begin{aligned}\text{Type II Error} &= \frac{FN}{TP + FN} \\ &= \frac{1}{3 + 1} \\ &= \frac{1}{4} = 0.25\end{aligned}$$

Type II Error = **25%**

Final Answers (for Cat class)

Metric	Value
Accuracy	80%
Precision	75%
Recall	75%
F1-score	75%
Specificity	83.33%
Type I Error	16.67%
Type II Error	25%

➤ **Shortcut: accuracy, precision, recall, f1-score, specificity, type 1 and 2 error for cat, dog and horse class**

Index	1	2	3	4	5	6	7	8	9	10
Actual	Cat	Dog	Horse	Cat	Dog	Horse	Cat	Dog	Horse	Cat
Predicted	Cat	Dog	Dog	Horse	Dog	Horse	Cat	Cat	Horse	Cat
Result	Correct	Correct	Misclassified	Misclassified	Correct	Correct	Correct	Misclassified	Correct	Correct

Multiclass Metrics Table

Metric	Cat	Dog	Horse
TP	3	2	2
FP	1	1	1
FN	1	1	1
TN	5	6	6
Accuracy	80%	80%	80%
Precision	75%	66.67%	66.67%
Recall (Sensitivity)	75%	66.67%	66.67%
F1-score	75%	66.67%	66.67%
Specificity	83.33%	85.71%	85.71%
Type I Error (FPR)	16.67%	14.29%	14.29%
Type II Error (FNR)	25%	33.33%	33.33%

- ✓ **TP (True Positive)**
Actual Cat and Predicted Cat = 3
- ✓ **FN (False Negative)**
Actual Cat but predicted something else = 1
- ✓ **FP (False Positive)**
Predicted Cat but actually not Cat
- ✓ **TN (True Negative)**
Everything except Cat-related errors/corrects

❑ Implementation of Confusion Matrix for Multi-Class classification using Python

Step 1: Import the necessary libraries

```
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
import matplotlib.pyplot as plt
```

Step 2: Create the NumPy array for actual and predicted labels

- y_true**: List of true labels.
- y_pred**: List of predicted labels by the model.
- classes**: A list of class names: 'Cat', 'Dog' and 'Horse'

```
y_true = ['Cat'] * 10 + ['Dog'] * 12 + ['Horse'] * 10
y_pred = ['Cat'] * 8 + ['Dog'] + ['Horse'] + ['Cat'] * 2 + ['Dog'] * 10 + ['Horse'] * 8 + ['Dog'] * 2
classes = ['Cat', 'Dog', 'Horse']
```

Step 3: Generate and Visualize the Confusion Matrix

- ConfusionMatrixDisplay**: Creates a display object for the confusion matrix.
- confusion_matrix=cm**: Passes the confusion matrix (cm) to display.
- display_labels=classes**: Sets the labels (['Cat', 'Dog', 'Horse']) or the confusion matrix.

```

cm = confusion_matrix(y_true, y_pred, labels=classes)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=classes)
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix', fontsize=15, pad=20)
plt.xlabel('Prediction', fontsize=11)
plt.ylabel('Actual', fontsize=11)
plt.gca().xaxis.set_label_position('top')
plt.gca().xaxis.tick_top()
plt.gca().figure.subplots_adjust(bottom=0.2)
plt.gca().figure.text(0.5, 0.05, 'Prediction', ha='center', fontsize=13)
plt.show()

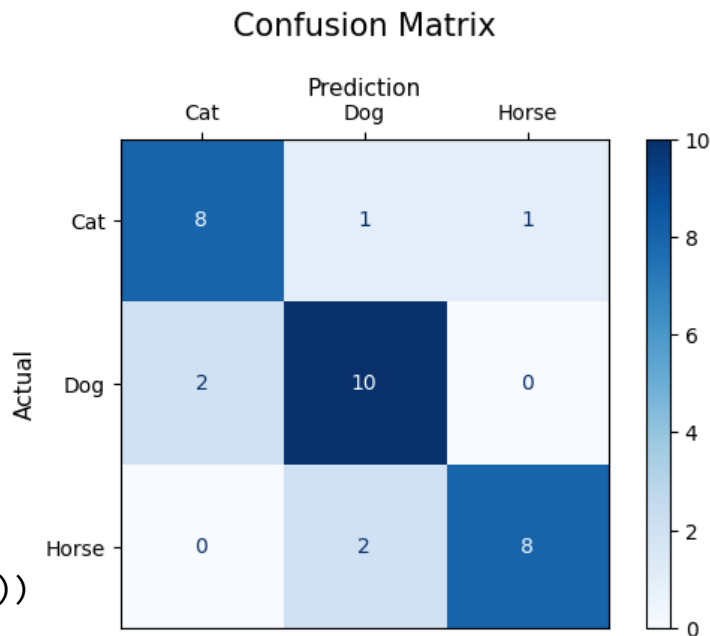
```

Step 4: Print the Classification Report

```
print(classification_report(y_true, y_pred, target_names=classes))
```

Output:

	precision	recall	f1-score	support
Cat	0.80	0.80	0.80	10
Dog	0.77	0.83	0.80	12
Horse	0.89	0.80	0.84	10
accuracy			0.81	32
macro avg	0.82	0.81	0.81	32
weighted avg	0.82	0.81	0.81	32



See you in the next lecture...